

Authmatic Team Handoff

Prior-Auth Killer | For backend / agent integration

This report summarizes what is built, what the mock HealthFirst payer portal expects from the agent, and what still needs to be wired (FastAPI agent, sponsor APIs). Demo patient: Sarah Martinez | Ozempic | HealthFirst PPO.

1. What we have (built)

- Demo PDFs: assets/demo/patient_chart_sarah_martinez.pdf + prescription_ozempic_martinez.pdf
- Mock data: mock/healthfirst-case.json (all extractable fields + workflow API paths)
- Next.js app: apps/web (runs on port 3000)
- Mock HealthFirst prior-auth form: /portal/healthfirst/prior-auth
- Realistic PA workflow - submit does NOT auto-approve:
 - Submit -> status pending_review -> agent calls adjudicate -> under_review -> approved/denied
- Status page (polls live): /portal/healthfirst/submission/{reference_id}
- Portal APIs (in apps/web, ready for agent to call):
 - POST /api/pa/submit -> creates PA, returns reference_id (pending)
 - GET /api/pa/{ref} -> current status + form payload
 - POST /api/pa/{ref}/adjudicate -> runs payer medical review (~8s) -> approved/denied
- Planning docs: spec.md, architecture.md, implementation.md, demo.md, risks.md
- Pitch deck: presentation.html (Opsera / Daytona / Insforge / Rtrvr 4-verb story)

2. Mock portal - form fields (Rtrvr must use these IDs)

Field ID	Sarah Martinez demo value
patient_name	Sarah Martinez
dob	03/14/1986
member_id	HF45821973
diagnosis	Type 2 Diabetes (E11.9)
medication	Ozempic
dosage	0.25mg weekly
provider_name	Emily Chen, MD
justification	Poor glycemic control despite first-line therapy

Submit button id: submit-prior-auth | Form action: POST /api/pa/submit

PORTAL_URL (set in .env): http://localhost:3000/portal/healthfirst/prior-auth

3. Agent loop - what backend must implement

apps/agent (FastAPI) does not exist yet - only README. Teammate builds this. Suggested steps aligned with pitch deck + mock portal:

Step	Verb	Sponsor	Action
1	READ-WEB	Rtrvr	Optional: fetch payer rules (or skip - we host mock portal)
2	EXECUTE	Daytona	Parse PDFs in sandbox (pdfplumber); fallback: healthfirst-case.json
3	VERIFY	Opsera	MCP scan_pii on outgoing packet before submit
4	READ-WEB	Rtrvr	Fill mock portal form + click Submit; capture reference_id from URL
5	API call	Web app	POST /api/pa/{ref}/adjudicate - triggers payer review
6	PERSIST	Insforge	Store run, agent_events, receipt_url, reference_id

Required FastAPI endpoints (apps/agent):

```
POST /api/run          <- dropzone uploads chart + rx PDFs, returns run_id
GET  /api/stream/{run_id} <- SSE: stream each agent step to UI
GET  /api/run/{run_id}  <- full audit payload for /run/{id} page
GET  /health           <- smoke test
```

4. Rtrvr task (copy-paste shape)

```
task = (
    f"Open {PORTAL_URL}. Fill the prior authorization form: "
    f"patient_name={fields['patient_name']}, "
    f"dob={fields['dob']}, member_id={fields['member_id']}, "
    f"diagnosis={fields['diagnosis']}, medication={fields['medication']}, "
    f"dosage={fields['dosage']}, provider_name={fields['provider_name']}, "
    f"justification={fields['justification']}. "
    "Click Submit Prior Authorization. "
    "Return the reference_id from the submission page URL."
)
# After Rtrvr returns reference_id:
# POST {WEB_URL}/api/pa/{reference_id}/adjudicate body: {"review_delay_ms": 8000}
```

5. What still needs to be built

YOU (frontend) - remaining:

- Dropzone UI on / (upload chart + prescription)
- /run/[id] audit page with SSE step cards + receipt banner
- Deploy apps/web to Render; set PORTAL_URL to public URL

TEAMMATE (backend) - primary:

- Scaffold apps/agent FastAPI service
- Wire Opsera MCP (VERIFY), Daytona (EXECUTE), Insforge (PERSIST)
- Wire Rtrvr (SUBMIT to mock portal)
- After Rtrvr submit: call POST /api/pa/{ref}/adjudicate on web app

- SSE stream + Insforge schema (prior_auths, agent_events)
- Deploy agent to Render; connect WEB_URL + AGENT_URL
- OPTIONAL: Tigris for PDF/receipt storage (stretch - Insforge storage also OK)

6. Sponsor API credentials

Copy to project root .env (from teammate). Fill in actual keys at hackathon:

```
OPSERA_MCP_URL=https://mcp.opsera.io/mcp
OPSERA_API_TOKEN=

DAYTONA_API_KEY=
DAYTONA_API_URL=https://api.daytona.io

INSFORGE_API_KEY=
INSFORGE_PROJECT_URL=

RTRVR_API_KEY=

# App URLs (add these)
WEB_URL=http://localhost:3000
AGENT_URL=http://localhost:8000
PORTAL_URL=http://localhost:3000/portal/healthfirst/prior-auth
DEMO_FIXTURE_MODE=true
```

7. Sponsor roles (pitch deck alignment)

Sponsor	Demo role
Rtrvr.ai	READ-WEB - drives browser, fills + submits mock HealthFirst form
Daytona	EXECUTE - sandbox PDF parse (pdfplumber, ICD-10 normalize)
Opsera	VERIFY - MCP scan_pii on outgoing packet (no PHI over-disclosure)
Insforge	PERSIST - Postgres workflow, agent_events, audit log
Render	Host web + agent publicly (live demo URL for judges)

8. End-to-end demo flow (judge view)

- 1. Upload Sarah chart + Ozempic prescription PDFs
- 2. Agent EXTRACTs fields (Daytona / LLM)
- 3. Opsera VERIFY green check on packet
- 4. Rtrvr fills HealthFirst form live (browser visible)
- 5. Submit -> Pending Review (NOT instant approve)
- 6. Agent adjudicates -> Under Review -> Approved
- 7. Audit page /run/{id}: receipt + full chain + sponsor logos

9. Repo map

```
authmatic/
  apps/web/          <- Next.js + mock portal (BUILT)
  apps/agent/        <- FastAPI agent (NOT BUILT - teammate)
  assets/demo/       <- Sarah PDFs
  mock/              <- healthfirst-case.json
```

```
presentation.html <- pitch deck  
docs/             <- this handoff PDF
```

10. Critical coordination

- PORTAL_URL must be the PUBLIC Render URL in prod - not localhost - or Rtrvr fails
- Field IDs on form must match exactly (see section 2)
- Submit != Approved. Agent must call /adjudicate after Rtrvr submit
- Member ID must be HF45821973 format or mock payer denies the PA
- Fallback: DEMO_FIXTURE_MODE=true + pre-recorded form-fill clip

Questions? Check architecture.md and apps/web/README.md in the repo.